Normalización y exploración de transacciones multifuente

Para desarrollar este proyecto, tomé varias decisiones sobre cómo debía funcionar la normalización de las transacciones provenientes de diferentes fuentes. Estas reglas las definí y configuré en el archivo config/rules.json, desde donde se aplican en el código.

1. Decisiones de diseño

Primero, definí la estructura que tendría el resultado final de cada transacción. Los campos principales son: id, amount, currency, timestamp, status y source. También agregué dos campos adicionales, original_amount y flags, para conservar información sobre posibles modificaciones realizadas durante la normalización.

Para manejar los montos utilicé Decimal internamente, ya que permite trabajar con valores monetarios sin los problemas de precisión que pueden aparecer al utilizar números de punto flotante. Al exportar los datos, el monto se convierte a un número. Los estados se mantienen en tres categorías: SUCCESS, FAILED y PENDING.

Para identificar de dónde proviene cada transacción, utilizo un campo identificador que puede ser id, transaction_id o ref, dependiendo de la fuente. Si no encuentro ninguno de estos campos, no es posible identificar correctamente el origen del registro, por lo que se envía a cuarentena con la marca UNKNOWN_SOURCE.

También tuve que adaptar el procesamiento de los montos según el formato de cada fuente:

- En gateway_a, los montos vienen como texto decimal.
- En gateway_b, los montos vienen como números enteros expresados en unidades menores. Por ejemplo, 10050 representa 100.50. La conversión depende de la cantidad de decimales que utiliza la moneda.
- En gateway_c, los montos vienen como texto, incluyendo el símbolo de la moneda y utilizando coma decimal, por ejemplo, "€99,99". En este caso, también se intenta identificar la moneda a partir del símbolo. Cuando aparece $, se asume que corresponde a USD.

En cuanto a las fechas, cada fuente tiene su propio formato. Por ejemplo, gateway_b utiliza el formato día/mes/año. Cuando una fecha no incluye información de zona horaria, asumí que corresponde a UTC. Finalmente, todas las fechas se convierten al formato ISO-8601 en UTC para mantener un formato uniforme.

Para los estados, establecí una equivalencia entre los diferentes nombres que pueden utilizar las fuentes y los tres estados permitidos: SUCCESS, FAILED y PENDING. Si aparece un estado que no está contemplado, el registro se considera inválido, ya que prefiero no asumir o inventar su significado.

También establecí algunas condiciones para determinar si una transacción es válida:

- El monto debe ser mayor que cero.
- El monto no puede superar 1,000,000.
- La moneda debe estar dentro de las 42 monedas soportadas.

- Se debe respetar la cantidad de decimales correspondiente a cada moneda.
- La fecha no puede ser futura.
- Todos los campos obligatorios deben estar presentes.
- No se permiten transacciones duplicadas, considerando como referencia la combinación de fuente e identificador. Si existe un duplicado, se conserva únicamente la primera aparición.

Los registros que no cumplen estas condiciones se envían a cuarentena, conservando tanto la información original como el motivo por el cual fueron rechazados. De esta manera, puedo mantener un control de todos los registros procesados y asegurar que el total de transacciones siempre corresponda a la suma de las válidas y las inválidas.

Por último, implementé una lógica de redondeo para los montos que tienen más decimales de los permitidos por su moneda. Si el exceso es de hasta tres decimales, el sistema realiza un redondeo utilizando ROUND_HALF_UP, conserva el monto original en original_amount y agrega la marca REQUIERE_REVISION. Además, registra el cambio en un archivo de auditoría ubicado en logs/audit_<dataset>.json, que también se muestra en una pestaña llamada Auditoría. Si el monto tiene más de tres decimales adicionales, se considera inválido y se envía a cuarentena.

2. Uso de inteligencia artificial (Claude, ChatGPT y Copilot)

Durante el desarrollo utilicé Claude y Copilot como herramientas de apoyo. Principalmente, me ayudaron a estructurar el código, proponer diferentes formas de procesar los datos y construir la lógica necesaria para interpretar los distintos formatos de montos, separadores decimales, separadores de miles y unidades menores. También me apoyaron en la construcción de la interfaz utilizando Flask.

Sin embargo, las decisiones principales sobre el funcionamiento del sistema fueron definidas por mí. Establecí la estructura final de las transacciones, las reglas de normalización y los criterios para determinar cuándo un registro debía considerarse válido o inválido. Todas estas decisiones quedaron reflejadas en el archivo rules.json.

Para comprobar que el sistema funcionara correctamente, realicé pruebas utilizando siete casos definidos en tests/ y dos archivos de datos: uno con información limpia y otro que contiene diferentes inconsistencias.

3. Cambios realizados a las sugerencias de la IA

Durante el desarrollo también realicé algunos cambios sobre las propuestas iniciales de la inteligencia artificial, ya que consideré que algunas reglas podían mejorarse.

El primer cambio fue en el tratamiento de los montos con más decimales de los permitidos. Inicialmente, la IA proponía invalidar cualquier transacción que presentara este problema. Sin embargo, decidí implementar un redondeo para los casos recuperables, siempre que el exceso no fuera mayor a tres decimales. De esta manera, evito perder transacciones que pueden corregirse, pero mantengo un registro del valor original y de los cambios realizados mediante la auditoría.

Otro cambio importante fue la cantidad de monedas soportadas. La propuesta inicial contemplaba únicamente cuatro monedas: USD, EUR, GBP y MXN. Decidí ampliar esta lista a 42 monedas, considerando la precisión decimal que utiliza cada una. Esto también implicó adaptar la conversión de unidades menores y el redondeo. Por ejemplo, el JPY utiliza cero decimales, el KWD utiliza tres y el CLF utiliza cuatro.

También decidí que REQUIERE_REVISION no debía manejarse como un estado de la transacción, sino como una marca adicional o flag. Esto es importante porque quería mantener los estados originales del sistema (SUCCESS, FAILED y PENDING) sin agregar categorías que modificaran esa estructura.

Finalmente, mantuve algunas decisiones de la propuesta original, como considerar inválidos los montos negativos o iguales a cero, rechazar los estados desconocidos, enviar los duplicados a cuarentena y utilizar UTC como zona horaria estándar.

4. Limitaciones del sistema

A pesar de las validaciones implementadas, el sistema tiene algunas limitaciones que deben considerarse.

Una de ellas es que algunos símbolos monetarios pueden ser ambiguos. Por ejemplo, el símbolo $ puede representar diferentes monedas y lo mismo sucede con ¥. Si la fuente no proporciona un código de moneda, el sistema no siempre puede identificarla con total seguridad.

Otra limitación es el tratamiento de las fechas que no incluyen zona horaria. En estos casos, el sistema asume que corresponden a UTC, aunque esto no necesariamente representa la zona horaria real en la que se realizó la transacción.

Por último, actualmente el sistema carga el conjunto completo de datos en memoria para procesarlo. Esto funciona para los archivos utilizados durante las pruebas, pero podría representar un problema de rendimiento si en el futuro se trabaja con cantidades mucho mayores de transacciones.